

If, If-else, Nested if

Conditional statements let your program make decisions and execute different code based on whether a condition is true or false.

When to use:

whenever your program needs to behave differently depending on a condition (pass/fail, eligible/not eligible, +ve/-ve).

Where to use:

Anywhere logic depends on a condition - inside main, inside methods, inside loops, "Nested if" is used when a decision depends on another decision already being true (a sub-condition).

How to use:

A. Simple if -

runs only if condition is true

Ex:

```
int age = 20;
if (age >= 18) {
    S.O.P("You can vote");
}
```

B. else-if -

One of two paths always runs.

Ex:

```
if (age >= 18) {
    S.O.P("Eligible");
} else {
    S.O.P("Not Eligible");
}
```

C. if-else-if-else -

multiple conditions checked in order.

Ex:

```
int marks = 75;
if (marks >= 90) {
    S.O.P("A");
} else if (marks >= 75) {
    S.O.P("B");
} else if (marks >= 50) {
    S.O.P("C");
} else {
    S.O.P("Fail");
}
```

D. Nested-if -

if inside another if.

Ex:

```
if (age >= 18) {
    if (hasID) {
        S.O.P("Entry allowed");
    } else {
        S.O.P("ID required");
    }
} else {
    S.O.P("Too young");
}
```

Why:

- * else if chain stops at the first true condition - One matters! Always check most specific/highest condition first.
- * Nested if = used when the second condition only makes sense after the first is true (a real dependency), unlike else if which checks independent alternatives.
- * Braces { } define what's inside the conditional block. Omitting them for single statements is legal but risky - a common bug when adding a second line later without adding braces.

Real-life Example:

Scenario

Rain → take umbrella
Have ticket → board; else
buy one
Grading system (A/B/C/Fail)
18+ AND has ID → enter
Traffic light logic

Structure

Simple if
if - else

else - if chain

Nested if

else if chain